

Trabajo Final Integrador (TFI)

Bases de Datos I

GRUPO 126

ALUMNOS

Matías Orellana - Comisión 2

Fernando Gómez - Comisión 14

Rodrigo Galván - Comisión 12

Nicolas Pagola - Comisión 9

Introducción

Este trabajo implementa un sistema de gestión comercial para comercio electrónico, modelando productos, marcas, categorías y códigos de barras. Se desarrolló un esquema robusto con constraints de integridad, se cargaron 500,000 registros mediante procedimientos almacenados, y se optimizaron consultas complejas con índices. Se implementaron medidas de seguridad con usuarios de privilegios mínimos y vistas para ocultar datos sensibles, además de manejo de transacciones con control de concurrencia y deadlocks.

Enlace al video de presentación:

<https://www.youtube.com/watch?v=ZNIK6E8gSwU>

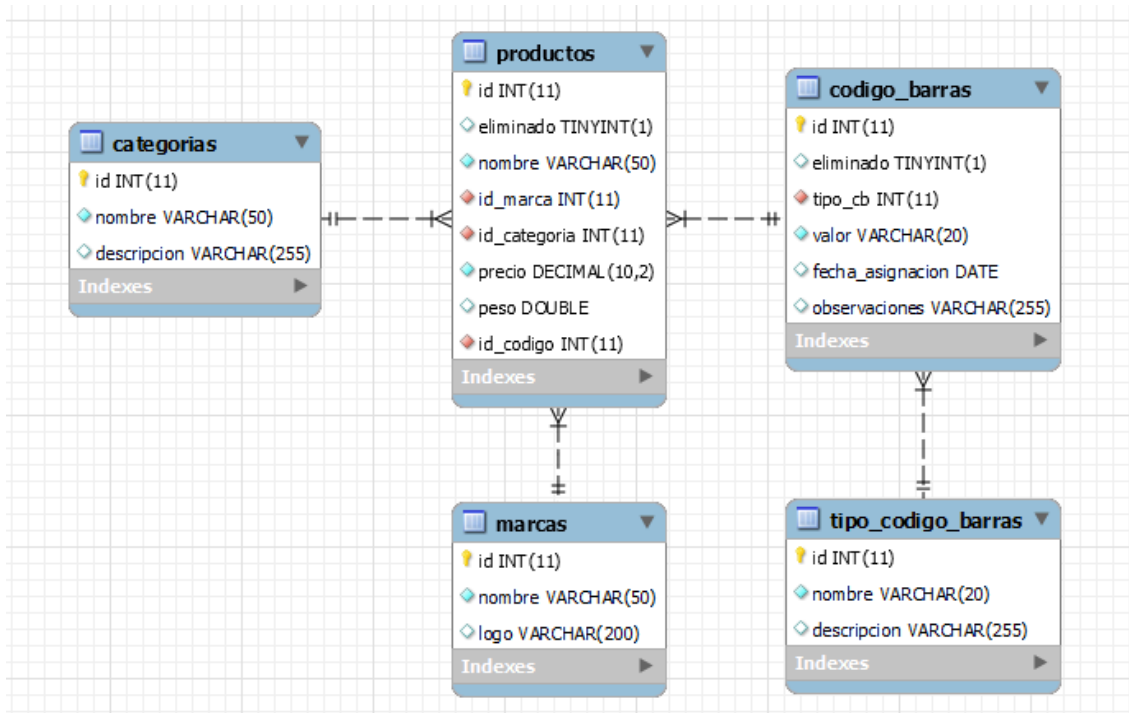
Etapas 1 – Modelado y Definición de Constraints

REGLAS DE NEGOCIO:

- **Dominio:** Sistema de gestión de productos para e-commerce.
- **Entidades principales:** Productos, Marcas, Categorías, Códigos de Barras.
- **Restricciones:**
 - Los productos deben tener precio ≥ 0 y ≤ 1000 .
 - Códigos de barras únicos por producto.
 - Integridad referencial entre tablas relacionadas.
 - Marcas y categorías predefinidas en catálogos.
- **Volumen:** 510,000 productos para pruebas de performance.

DER:

Modelo entidad-relación con 5 entidades principales y relaciones definidas por claves foráneas.



Creación de la base de datos

Para pasar del diagrama a la base de datos, se utilizó MySQL como SGBD y se escribieron los comandos necesarios en el script 01_esquema.sql.

Validación de inserciones:

Inserciones correctas:

```
INSERT INTO marcas (nombre_marca) VALUES ('Nike'), ('Adidas');
```

✓ 1 19:30:14 INSERT INTO marcas (nombre_marca) VALUES ('Nike'), (... 2 row(s) affected Records: 2 Duplicates: 0 Warnings: 0

Inserciones erróneos:

```
INSERT INTO marcas (nombre_marca) VALUES ('Samsung'); -- Error: duplicate entry
```

✗ 1 19:23:20 INSERT INTO marcas (nombre_marca) VALUES ('Samsung') Error Code: 1062. Duplicate entry 'Samsung' for key 'nombre_marca'

```
INSERT INTO productos (nombre_producto, marca_id, categoria_id, precio, codigo_id)
VALUES ('Producto Test', 999, 999, 100.00, 96565699); -- Error: constraint foreign key
```

✗ 2 19:27:01 INSERT INTO productos (nombre_producto, marca_id, cat... Error Code: 4025. CONSTRAINT 'productos.marca_id' failed for 'gestion_comercio'. 'productos'

Decisiones de Diseño y Constraints

Estrategia de Claves Primarias:

- Se utilizaron claves primarias auto-incrementales (AUTO_INCREMENT) para garantizar unicidad y simplificar las inserciones
- Todas las tablas principales incluyen un ID numérico como identificador único

Constraints de Integridad Implementados:

- CHECK constraints para validar rangos: precio ≥ 0 , tipo_id entre 1-3, marca_id entre 1-20.
- UNIQUE constraints** en campos críticos: código de barras, nombres de marcas y categorías.
- NOT NULL en atributos obligatorios para garantizar completitud de datos
- FOREIGN KEY constraints con referencias completas entre todas las tablas relacionadas

Estrategia de Eliminación Lógica

- Campos `eliminado BOOLEAN` en tablas principales para mantener historial y permitir recuperación
- Consultas filtran por `eliminado = false` para excluir registros dados de baja
- Permite auditoría y cumplimiento de regulaciones de retención de datos

Diseño de Relaciones

- Relación 1:1 entre productos y códigos de barras (un producto, un código único)
- Relación N:1 entre productos y marcas/categorías (múltiples productos por marca/categoría)
- Catálogos predefinidos (marcas, categorías, tipos de código) para garantizar consistencia

Estrategia de Carga Masiva

- Inserción ordenada: catálogos → códigos de barras → productos
- Uso de LAST_INSERT_ID() para mantener integridad referencial automáticamente
- Procedimientos almacenados para generación reproducible de datos

Evidencia de interacción con IA

Ver AnexoIA_TFI_BDI_Comision2_Grupo126_Orellana-Gomez-Galvan-Pagola.pdf
Etapa 1

Etapa 2. Generación y carga de datos masivos con SQL puro

Descripción conceptual del mecanismo de poblamiento

Para poblar la base de datos de forma automática, se optó por el uso de series o secuencias numéricas como mecanismo para la generación de datos. Para esto se diseñó un procedimiento almacenado (INSERTAR_CODIGO_BARRA) controlado mediante un contador (i), combinado con generación aleatoria para ciertos atributos. Este mecanismo tiene como objetivo crear registros únicos y consistentes en las tablas codigo_barras y productos, utilizando una lógica incremental.

La lógica del procedimiento es la siguiente:

- Se define un bucle WHILE controlado por una variable contador (i) que itera hasta alcanzar la cantidad de registros solicitados.
- Dentro de cada iteración:
 - Se genera un valor único y secuencial para el código de barras, utilizando la función LPAD sobre el contador i, lo que garantiza la unicidad del campo valor (por ejemplo: PROD00000001, PROD00000002, etc.).
 - Se asigna aleatoriamente un tipo de código de barras (tipo_cb) utilizando RAND(), dentro de un rango definido.
 - Se inserta el registro en la tabla codigo_barras, mediante LAST_INSERT_ID(), se recupera el id generado automáticamente para mantener la integridad referencial con la tabla productos.
 - Para el producto relacionado, se generan el resto de valores:
 - Nombre secuencial (PRODUCTAZO1, PRODUCTAZO2, ...) combinando un string base con el contador.
 - Precio y peso aleatorios dentro de un rango definido.
 - Claves foráneas aleatorias para marca y categoría, representando la distribución de productos en diferentes agrupaciones.

Este enfoque permite reutilizar el procedimiento con distintas cantidades de registros.

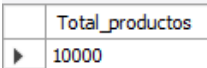
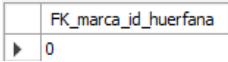
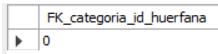
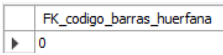
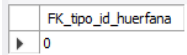
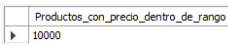
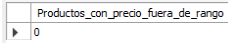
Para respetar las cardinalidades entre las diferentes tablas, se definieron claves foráneas en las tablas dependientes que apuntan a las claves primarias de sus tablas relacionadas. Además, durante el poblamiento, se insertaron los registros en un orden que asegura que siempre existan los valores referenciados, garantizando así la integridad referencial.

Implementación:

Ver 02_catalogos.sql: Inserción de datos maestros (marcas, categorías, tipos de código).

Ver 03_carga_masiva.sql: Procedimiento almacenado insertar_codigo_barra y llamadas para generación de 510,000 registros.)

Pruebas de consistencia

Verificación	Consulta	Respuesta	Observaciones
Conteos	<pre>SELECT COUNT(*) as Total_productos FROM productos;</pre>		Esta consulta devuelve el total de registros en la tabla productos
FK huérfanas: Marca	<pre>SELECT COUNT(p.producto_id) as FK_marca_id_huerfana FROM productos p LEFT JOIN marcas m ON p.marca_id = m.marca_id WHERE m.marca_id IS NULL;</pre>		Si devuelve algún valor NULL quiere decir que hay un producto con una marca que no se encuentra en la tabla marcas.
FK huérfanas: Categoría	<pre>SELECT COUNT(p.producto_id) as FK_categoria_id_huerfana FROM productos p LEFT JOIN categorias c ON p.categoria_id = c.categoria_id WHERE c.categoria_id IS NULL;</pre>		Si devuelve algún valor NULL quiere decir que hay un producto con una categoria_id que no se encuentra en la tabla categorias.
FK huérfanas: Código de barras	<pre>SELECT COUNT(p.producto_id) as FK_codigo_barras_huerfana FROM productos p LEFT JOIN codigoi_barras cb ON p.codigo_id = cb.codigo_id WHERE cb.codigo_id IS NULL;</pre>		Si devuelve algún valor NULL quiere decir que hay un producto con una codigo_id que no se encuentra en la tabla codigo_barras.
FK huérfanas: Tipo de código de barras	<pre>SELECT COUNT(cb.codigo_id) as FK_tipo_id_huerfana FROM codigo_barras cb LEFT JOIN tipo_codigoi_barras tcb ON cb.tipo_id = tcb.tipo_id WHERE tcb.tipo_id IS NULL;</pre>		Si devuelve algún valor NULL quiere decir que hay un codigo de barras con un tipo_id que no se encuentra en la tabla tipo_codigo_barras.
Rango de precio 0.00 a 1000.00	<pre>SELECT COUNT(*) as Productos_con_precio_dentro_de_rango FROM productos p WHERE p.precio > 0 && p.precio < 1000;</pre>		Consulta con condición de precio dentro del rango.
Rango de precio fuera de 0.00 a 1000.00	<pre>SELECT COUNT(*) as Productos_con_precio_fuera_de_rango FROM productos p WHERE p.precio < 0 && p.precio > 1000;</pre>		Consulta con condición de precio fuera del rango:

Etapa 3. Consultas complejas y útiles a partir del CRUD inicial

Consultas con JOIN

Consulta 1:

La utilidad de esta consulta para el negocio radica en poder presentar una lista de los productos, su código de barra, su marca, categoría y precio. Se puede obtener una vista general sobre el producto y filtrar tan solo con modificar la cláusula Where, dependiendo de si se necesita que la búsqueda sea más específica a marcas, precios, categorías o un rango de códigos de barra.

```
SELECT
    p.nombre_producto as Nombre,
    c.valor as Codigo_Barra,
    m.nombre_marca,
    cat.nombre_categoria,
    p.precio as Precio
FROM productos as p
JOIN codigo_barras as c
ON p.codigo_id = c.codigo_id
JOIN marcas as m
ON p.marca_id = m.marca_id
JOIN categorias as cat
ON p.categoria_id = cat.categoria_id
WHERE p.eliminado = false AND c.eliminado = false
ORDER BY p.nombre_producto ASC;
```

```
CREATE INDEX idx_productos_nombre ON productos(nombre_producto);
```

Tabla de tiempos:

	Corrida 1(s)	Corrida 2(s)	Corrida 3(s)	Mediana(s)
SIN ÍNDICE	3.234	3.828	3.843	3.828
CON ÍNDICE	3.875	3.718	3.890	3.875

En esta primera ejecución solo con el índice en “nombre_producto” no se observa un cambio debido a que probablemente el cuello de botella que “retrasa” la consulta no es exclusivamente el campo “nombre_producto”. Por esta razón se aplicó un índice compuesto por “nombre_producto” y “eliminados” (otro campo que se evalúa) de la tabla productos para ver qué sucede.

	Ejecución 1(s)	Ejecución 2(s)	Ejecución 3(s)	Mediana(s)
CON ÍNDICE COMPUESTO	3.110	3.375	3.062	3.110

En este caso, se observa un cambio notable de alrededor del 20% de mejora en el tiempo de consulta, corroborando la premisa de que el factor limitante no era solamente "nombre_producto".

Consulta con JOIN (extraída de la consulta 1)

EXPLAIN consulta sin índice:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	c	ref	PRIMARY,FK_tipo_codigo	FK_tipo_codigo	4	const	232397	Using where; Using temporary; Using filesort
1	SIMPLE	t	const	PRIMARY	PRIMARY	4	const	1	
1	SIMPLE	p	eq_ref	codigo_id	codigo_id	4	gestion_comercio.c.codigo_id	1	Using where

EXPLAIN consulta con índice en nombre_producto:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	c	ref	PRIMARY,FK_tipo_codigo	FK_tipo_codigo	4	const	232397	Using where; Using temporary; Using filesort
1	SIMPLE	t	const	PRIMARY	PRIMARY	4	const	1	
1	SIMPLE	p	eq_ref	codigo_id	codigo_id	4	gestion_comercio.c.codigo_id	1	Using where

EXPLAIN consulta con índice compuesto en nombre_producto y eliminados:

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	p	ref	codigo_id,idx_productos_eliminado_nombre	idx_productos_eliminado_nombre	2	const	246943	Using temporary; Using filesort
1	SIMPLE	c	eq_ref	PRIMARY,FK_tipo_codigo	PRIMARY	4	gestion_comercio.p.codigo_id	1	Using where
1	SIMPLE	t	const	PRIMARY	PRIMARY	4	const	1	

EXPLAIN consulta con índice anterior y creamos índice en "tipo_id" en codigo_barras:

No hay una mejora de tiempo en este caso aunque SQL si hace uso del índice.

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	p	ref	codigo_id,idx_productos_eliminado_nombre	idx_productos_eliminado_nombre	2	const	246943	Using temporary; Using filesort
1	SIMPLE	c	eq_ref	PRIMARY,idx_codigo_barras_tipo_id	PRIMARY	4	gestion_comercio.p.codigo_id	1	Using where
1	SIMPLE	t	const	PRIMARY	PRIMARY	4	const	1	

Consulta 2:

Consulta para evaluar la presencia de referencias NULL y validar la integridad del sistema.

A fines de que sea apreciable, se limitó la consulta por tipo de código de barra

y se agregó un contador para ver el total de registros de la consulta.


```
SELECT
    p.nombre_producto as Producto,
    c.valor as Codigo_Barra,
    t.nombre as Tipo_Codigo,
    COUNT(*) OVER() as TOTAL -- over evita
    -- que se unifique la cuenta en una sola fila
    -- y la agrega como una columna nueva en cada tupla
FROM productos as p
LEFT JOIN codigo_barras as c
ON p.codigo_id = c.codigo_id
LEFT JOIN tipo_codigo_barras as t
ON c.tipo_id = t.tipo_id
WHERE p.eliminado = false
and c.eliminado = false
AND c.tipo_id = 1
ORDER BY p.nombre_producto;
```

Resultado:

Producto	Codigo_Barra	Tipo_Codigo	TOTAL
PRODUCTAZO100059	PROD00100059	EAN13	112295
PRODUCTAZO100062	PROD00100062	EAN13	112295
PRODUCTAZO100063	PROD00100063	EAN13	112295
PRODUCTAZO100065	PROD00100065	EAN13	112295
PRODUCTAZO100073	PROD00100073	EAN13	112295
PRODUCTAZO100074	PROD00100074	EAN13	112295
PRODUCTAZO100076	PROD00100076	EAN13	112295
PRODUCTAZO100078	PROD00100078	EAN13	112295
PRODUCTAZO100080	PROD00100080	EAN13	112295

Consulta con Where

Esta consulta retorna todos los productos marca Apple cuyo precio sea 399.

```
SELECT * FROM productos
WHERE precio = 399 -- Apple
AND eliminado = false;
```

Tabla de tiempos:

	Corrida 1(s)	Corrida 2(s)	Corrida 3(s)	Mediana(s)
SIN ÍNDICE	1.844	2.204	2.312	2.204
CON ÍNDICE	0.016	0.000	0.000	0.000

EXPLAIN sin índice

	id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
▶	1	SIMPLE	productos	ref	idx_productos_eliminado_nombre	idx_productos_eliminado_nombre	2	const	246943	Using where

EXPLAIN con índice en la columna “precio” de la tabla productos.

```
CREATE INDEX idx_productos_precios ON productos(precio);
```

	id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
▶	1	SIMPLE	productos	ref	idx_productos_eliminado_nombre,idx_productos_precios	idx_productos_precios	5	const	7	Using where

Consulta con BETWEEN

```
SELECT * FROM productos
WHERE precio BETWEEN 300.10 AND 300.15
AND eliminado = false;
```

Tabla de tiempos:

	Corrida 1(s)	Corrida 2(s)	Corrida 3(s)	Mediana(s)
SIN ÍNDICE	0.234	0.250	0.266	0.250
CON ÍNDICE	0.000	0.000	0.000	0.000

EXPLAIN sin indice

	id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
▶	1	SIMPLE	productos	ALL	<u>NONE</u>	<u>NONE</u>	<u>NONE</u>	<u>NONE</u>	493887	Using where

EXPLAIN con índice en los campos “precio” y “eliminado” de la tabla productos.

```
CREATE INDEX idx_productos_precio_eliminado_1
ON productos(precio, eliminado);
```

	id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
▶	1	SIMPLE	productos	range	idx_productos_precio_eliminado_1	idx_productos_precio_eliminado_1	7	<u>NONE</u>	26	Using index condition

Consultas con GROUP BY + HAVING

En esta consulta se utilizó GROUP BY + HAVING y se realizaron 2 consultas.

La primera para obtener las marcas que en promedio tengan productos más caros.

La consulta 1 detecta productos de gama alta o productos muy caros sin ser de gama alta a la que se le pueda aplicar una promoción o descuento.

Por otro lado, en la consulta 2, simplemente se cambia el valor y el operador lógico de Having para obtener marcas de productos de gama baja respecto a precios.

La importancia de estas consultas para el negocio radica en que es útil para detectar qué marcas son más caras y cuáles más económicas y en qué rango de precios oscilan, para tomar decisiones sobre cómo ofertarlas o diseñar una estrategia de negocio en función de estas.

Consulta 1:

```
SELECT
    m.nombre_marca as Marca,
    COUNT(*) as cantidad_productos,
    AVG(p.precio) as precio_promedio
FROM marcas as m
JOIN productos as p ON p.marca_id = m.marca_id
WHERE p.eliminado = false
GROUP BY m.nombre_marca
HAVING AVG(p.precio) > 750;
```

Respuesta:

Marca	cantidad_productos	precio_promedio
Apple	10152	798.707114
DELL	10174	801.253461
Drean	10248	800.914026
HP	10241	799.635990
JBL	10344	800.448425
Nespresso	10212	799.822881
Samsung	10182	799.068535
Sony	10072	801.118421

Consulta 2:

```
SELECT
    m.nombre_marca as Marca,
    COUNT(*) as cantidad_productos,
    AVG(p.precio) as precio_promedio
FROM marcas as m
JOIN productos as p ON p.marca_id = m.marca_id
WHERE p.eliminado = false
GROUP BY m.nombre_marca
HAVING AVG(p.precio) < 400;
```

Respuesta:

Marca	cantidad_productos	precio_promedio
Liliana	12746	251.720373
PHILIPS	12860	248.771216

Consulta con subconsulta:

En esta consulta se realizó una operación del tipo analítica para lograr representar la cantidad relativa de productos por marcas y mostrar su frecuencia relativa acumulada a fines de poder realizar análisis respecto a la distribución de los productos, stock y cantidad por marca.

Consulta 1:

```
SELECT
    m.nombre_marca as Marca,
    COUNT(*) as Cant_Prod,
    CONCAT(ROUND(COUNT(*) * 100.0 / (SELECT COUNT(*)
    FROM productos WHERE eliminado = false), 2), '%') as Porcentaje,
    CONCAT(ROUND(SUM(COUNT(*) OVER(ORDER BY COUNT(*) ASC) * 100.0 / (SELECT COUNT(*)
    FROM productos WHERE eliminado = false), 2), '%') as Frecuencia_Relativa_Acumulada
FROM marcas as m
JOIN productos as p ON p.marca_id = m.marca_id
WHERE p.eliminado = false
GROUP BY m.nombre_marca
ORDER BY Porcentaje ASC;
```

Respuesta:

Marca	Cant_Prod	Porcentaje	Frecuencia_Relativa_Acumulada
Sony	10072	2.99%	2.99%
Apple	10152	3.02%	6.01%
DELL	10174	3.02%	9.03%
Samsung	10182	3.02%	12.06%
Nespresso	10212	3.03%	15.09%
HP	10241	3.04%	18.13%
Drean	10248	3.04%	21.18%
JBL	10344	3.07%	24.25%
Liliana	12746	3.79%	28.03%
PHILIPS	12860	3.82%	31.86%
Oster	25354	7.53%	39.39%
Electrolux	25438	7.56%	54.50%
Bosch	25436	7.56%	46.94%
Moulinex	25498	7.57%	77.21%
GA.MA	25475	7.57%	69.63%
Mourix	25467	7.57%	62.07%
Gafa	25510	7.58%	84.79%
LENOVO	25583	7.60%	92.39%
LG	25628	7.61%	100.00%

Vista productos_apple_disponibles:

La utilidad de esta vista reside en poder acceder sin demora al estado de los productos Apple que hay en disponibilidad sin tener que crear la consulta cada vez.

Es un activo estratégico el acceso a la vista porque son productos de gama alta y con gran movimiento actualmente que requieren consultas frecuentes.

```
CREATE VIEW productos_apple_disponibles AS
SELECT
    m.nombre_marca as Marca,
    cat.nombre_categoria as Categoria,
    COUNT(*) as Cantidad,
    p.nombre_producto as Producto,
    c.valor as CODIGO
FROM productos as p
JOIN codigo_barras as c ON p.codigo_id = c.codigo_id
JOIN marcas as m ON m.marca_id = p.marca_id
JOIN categorias as cat ON cat.categoria_id = p.categoria_id
WHERE m.nombre_marca LIKE 'Apple'
AND p.eliminado = false
GROUP BY Categoria;
```

Consulta:

```
SELECT * FROM productos_apple_disponibles ORDER BY Cantidad;
```

Respuesta:

Marca	Categoria	Cantidad	Producto	CODIGO
Apple	Herramientas y jardineria	149	PRODUCTAZO1593	PROD00001593
Apple	Computación	155	PRODUCTAZO3936	PROD00003936
Apple	Cuidado Personal	160	PRODUCTAZO1208	PROD00001208
Apple	Audio	174	PRODUCTAZO366	PROD00000366
Apple	Pequeños Electrodomésticos	193	PRODUCTAZO2370	PROD00002370
Apple	Celulares	1842	PRODUCTAZO27	PROD00000027
Apple	Climatización	1843	PRODUCTAZO121	PROD00000121
Apple	Consolas	1853	PRODUCTAZO704	PROD00000704
Apple	Linea Blanca	1858	PRODUCTAZO844	PROD00000844
Apple	Televisores	1925	PRODUCTAZO350	PROD00000350

Implementación:

Ver 05_consultas.sql: Consultas 1 (JOIN), 2 (LEFT JOIN), 3 (GROUP BY + HAVING) y 4 (Subconsulta).

Ver 06_vistas.sql: Creación de vista productos_apple_disponibles.

Ver 05_explain.sql y (04_indices.sql): Pruebas de performance con/sin índices

Etapa 4. Seguridad e integridad

Creación de usuario 'vendedor' y posterior consulta sobre sus permisos.

```
CREATE USER 'vendedor'@'localhost' IDENTIFIED BY '123456'; -- el usuario nuevo por defecto nace con permisos minimos  
  
SHOW GRANTS FOR 'vendedor'@'localhost'; -- lo verificamos
```

Gants del usuario al momento de la creación. Al final se mostrará el estado final de sus accesos.

	Grants for vendedor@localhost
►	GRANT USAGE ON *.* TO `vendedor`@`localhost` IDENTIFIED BY PASSWORD '*68B4837EB74329105EE4568DDA7DC6...

```
-- la seleccion da acceso  
use gestion_comercio;  
  
-- consulta con error, no hay base de datos seleccionada.  
Select * from productos;
```

Devuelve error de acceso usando USE.

Error Code: 1044. Access denied for user 'vendedor'@'localhost' to database 'gestion_comercio' 0.000 sec

Como no selecciona DB no puede ejecutar Select.

Error Code: 1046. No database selected Select the default DB to be used by double-clicking its name in the SCHEMAS list in the sidebar. 0.000 sec

```
-- Consulta vista  
  
SELECT * FROM celulares_disponibles_vendedor;
```

Todavía sin acceso da el error:

Error Code: 1046. No database selected Select the default DB to be used by double-clicking its name in the SCHEMAS list in the sidebar. 0.000 sec

Permisos de acceso otorgados

```

1  -- permisos de vista otorgados
2  -- la seleccion da acceso se puede ejecutar
3  * use gestion_comercio;
4
5  -- Consulta vistas ahora funciona para el usuario porque se le dio acceso desde el usuario root
6
7  * SELECT * FROM productos_ultimos_30_dias_vendedor;
8
9  * SELECT * FROM celulares_disponibles_vendedor;
10 --
11 -- consulta con error porque no tiene acceso
12 * Select * from productos;

```

	ID_Prod	Producto	Marca	Categoria	Fecha_Carga
▶	97	PRODUCTAZO97	Apple	Celulares	2025-10-10
	284	PRODUCTAZO284	Apple	Climatización	2025-10-07
	303	PRODUCTAZO303	Apple	Climatización	2025-10-13
	350	PRODUCTAZO350	Apple	Televisores	2025-10-03
	366	PRODUCTAZO366	Apple	Audio	2025-09-23
	407	PRODUCTAZO407	Apple	Televisores	2025-09-20

	ID_Prod	Producto	Marca	Categoria
▶	27	PRODUCTAZO27	Apple	Celulares
	97	PRODUCTAZO97	Apple	Celulares
	126	PRODUCTAZO126	Apple	Celulares
	135	PRODUCTAZO135	Apple	Celulares
	152	PRODUCTAZO152	Apple	Celulares
	762	PRODUCTAZO762	Apple	Celulares

```

-- Insert en un producto con marca_id que no existe
INSERT INTO productos (eliminado, nombre_producto, marca_id, categoria_id, precio, codigo_id)
VALUES (false, 'Producto Test FK', 999, 1, 100.00, 1);
-- Error Code: 4025. CONSTRAINT `productos.marca_id` failed for `gestion_comercio`.`productos` 0.000 sec

```

```

-- Intento duplicar un valor único en codigo_barras
INSERT INTO codigo_barras (eliminado, tipo_id, valor, fecha_asignacion)
VALUES (false, 1, 'PROD00000001', CURDATE());
-- Error Code: 1062. Duplicate entry 'PROD00000001' for key 'valor' 0.016 sec

```

```

-- Intentamos precio negativo
INSERT INTO productos (eliminado, nombre_producto, marca_id, categoria_id, precio, codigo_id)
VALUES (false, 'Producto Precio Negativo', 1, 1, -50.00, 2);
-- Error Code: 4025. CONSTRAINT `productos.precio` failed for `gestion_comercio`.`productos` 0.000 sec

```

```

-- Intentar duplicar PK (usando INSERT IGNORE o similar)
INSERT INTO marcas (marca_id, nombre_marca)
VALUES (1, 'Marca Duplicada');
-- Error Code: 1062. Duplicate entry '1' for key 'PRIMARY' 0.000 sec

```


- 1 18:51:17 INSERT INTO productos (eliminado, nombre_producto, marca_id, categoria_id, precio, codigo_id) VALUES (false, 'Producto Test FK', 999, 1, 100.00, 1) Error Code: 4025. CONSTRAINT productos.marca_id' failed for 'gestion_comercio'.productos'
- 2 18:51:19 INSERT INTO codigo_barras (eliminado, tipo_id, valor, fecha_asignacion) VALUES (false, 1, 'PROD00000001', CURDATE()) Error Code: 1062. Duplicate entry 'PROD00000001' for key 'valor'
- 3 18:51:20 INSERT INTO productos (eliminado, nombre_producto, marca_id, categoria_id, precio, codigo_id) VALUES (false, 'Producto Precio Negativo', 1, 1, -50... Error Code: 4025. CONSTRAINT productos.precio' failed for 'gestion_comercio'.productos'
- 4 18:51:23 INSERT INTO marcas (marca_id, nombre_marca) VALUES (1, 'Marca Duplicada') Error Code: 1062. Duplicate entry '1' for key 'PRIMARY'

```
SELECT TABLE_NAME, CONSTRAINT_TYPE, CONSTRAINT_NAME
FROM INFORMATION_SCHEMA.TABLE_CONSTRAINTS
WHERE TABLE_SCHEMA = 'gestion_comercio';
```

TABLE_NAME	CONSTRAINT_TYPE	CONSTRAINT_NAME
categorias	PRIMARY KEY	PRIMARY
categorias	UNIQUE	nombre_categoria
codigo_barras	PRIMARY KEY	PRIMARY
codigo_barras	UNIQUE	valor
codigo_barras	CHECK	tipo_id
codigo_barras	FOREIGN KEY	FK_tipo_codigo
marcas	PRIMARY KEY	PRIMARY
marcas	UNIQUE	nombre_marca
productos	PRIMARY KEY	PRIMARY
productos	UNIQUE	codigo_id
productos	CHECK	nombre_producto
productos	CHECK	marca_id
productos	CHECK	categoria_id
productos	CHECK	precio
productos	CHECK	peso
productos	FOREIGN KEY	FK_categoria
productos	FOREIGN KEY	FK_codigo_barras
productos	FOREIGN KEY	FK_marca
tipo_codigo_...	PRIMARY KEY	PRIMARY
tipo_codigo_...	UNIQUE	nombre

Procedimiento SQL no dinámico

Se implementó el procedimiento seguro `consultar\_precio\_producto` para conseguir el precio de un producto mediante la ID. Se llama mediante CALL desde el usuario, previo garantizar acceso desde el root.

Se implementó el procedimiento seguro `consultar\_precio\_producto` -

Ver 07_seguridad.sql (para la definición del procedimiento y la asignación de permisos al usuario 'vendedor')

```
DELIMITER $$
CREATE PROCEDURE consultar_precio_producto(IN ID INT)
BEGIN
    SELECT
        p.producto_id as ID,
        p.nombre_producto as Producto,
        c.valor as Codigo_Barra,
        m.nombre_marca as Marca,
        cat.nombre_categoria as Categoria,
        CONCAT('$',p.precio) as Precio
    FROM productos as p
    INNER JOIN codigo_barras as c ON c.codigo_id = p.codigo_id
    INNER JOIN marcas as m ON m.marca_id = p.marca_id
    INNER JOIN categorias as cat ON cat.categoria_id = p.categoria_id
    WHERE p.eliminado = false
    AND c.eliminado = false
    AND p.producto_id = ID;
END $$
DELIMITER ;

GRANT EXECUTE ON PROCEDURE consultar_precio_producto TO 'vendedor'@'localhost';
```

Ejecución del procedimiento en el usuario 'vendedor'.

```
15 -- ejecutamos el procedure
16
17 * CALL consultar_precio_producto(1);
18
19
```

ID	Producto	Codigo_Barra	Marca	Categoria	Precio
1	PRODUCTAZO1	PROD00000001	Gafa	Cuidado Personal	\$36.97

Accesos garantizados al usuario 'vendedor' al final de la Etapa 4.

	Grants for vendedor@localhost
▶	GRANT USAGE ON *,* TO `vendedor`@`localhost` IDENTIFIED BY PASSWORD '*6BB4837EB74329105EE4568DDA7DC6...
	GRANT SELECT ON `gestion_comercio`,`celulares_disponibles_vendedor` TO `vendedor`@`localhost`
	GRANT SELECT ON `gestion_comercio`,`productos_ultimos_30_dias_vendedor` TO `vendedor`@`localhost`
	GRANT EXECUTE ON PROCEDURE `gestion_comercio`,`consultar_precio_producto` TO `vendedor`@`localhost`

Prueba de seguridad anti-inyección

En esta etapa se deja constancia de las ejecuciones de consultas con intento de inyección SQL. El procedimiento cumple con los requisitos necesarios para evitar este tipo de consultas.

Los niveles de seguridad van desde el parseo al tipar el parámetro del procedimiento hasta restricciones de permisos de DDL.

En estos primeros ejemplos se observa cómo el compilador parsea el argumento y en el primer CALL corta el parseo en el primer carácter no numérico que aparece, devolviendo solo el equivalente a la consulta CALL **consultar_precio_producto(999)**.

En la segunda, también se filtra en el parseo pero no se ejecuta la consulta o devuelve vacío porque no puede parsear caracteres a números.

```
-- prueba de inyeccion
CALL consultar_precio_producto('999 OR 1=1'); -- devuelve consulta para id 999

CALL consultar_precio_producto('ABC OR 1=1'); -- esta consulta no parsea correctamente los caracteres y devuelve consulta vacia.
-- el parametro tipado en INT fueza el parseo y bloquea cualquier tipo de inyeccion.
```

La siguiente consulta es más reveladora porque ejecutando desde el usuario:

CALL consultar_precio_producto('1; DROP TABLE productos;');

Devuelve Error Code: 1370. execute command denied to user 'vendedor'@'localhost' for routine 'gestion_comercio.consultar_precio_producto'. Esto indica que el motor está detectando que el CALL tiene en su argumento sentencias que no está autorizado a usar. Esto en sí mismo no es una prueba correcta anti-inyección porque daría a entender que el motor sí está interpretando las sentencias después del parseo y, por un lado, estaría contradiciendo las revelaciones de las 2 consultas anteriores respecto al parseo INT truncando en caracteres no numéricos, y por otro lado, no da a entender que se intenta ejecutar la sentencia DDL. Por eso se va a ejecutar la misma consulta desde el root para saber si la sentencia DDL se ejecutaría si el usuario tuviera accesos.

```
CALL consultar_precio_producto('1; DROP TABLE productos;'); --  
-- esta ultima consulta no permite el paso ni siquiera al parse
```

A continuación, se puede ver el resultado de la consulta con sentencias DDL en el argumento intentando inyección SQL. La sentencia: CALL consultar_precio_producto('1; DROP TABLE productos;'); desde el root permite observar que el motor primero interpreta el contenido antes de siquiera parsearlo y por eso detectaba que el contenido tenía palabras reservadas para DDL y ni siquiera parseaba el contenido a INT.

Ahora desde el root se ve la misma conducta que desde el usuario, el contenido se parsea a pesar de las palabras reservadas para DDL y cuando se parsea trunca en el primer caracter no numérico: en este caso “ “.

129 • `CALL consultar_precio_producto('1; DROP TABLE productos;');`

Result Grid

Filter Rows:

Export: 

Wrap Cell Content: 

	ID	Producto	Codigo_Barra	Marca	Categoria	Precio
▶	1	PRODUCTAZO1	PROD00000001	Gafa	Cuidado Personal	\$36.97

Si desde el usuario se hace el call con '1' se obtiene el mismo resultado que haciendo el call con 1. Lo que valida el razonamiento del parseo a INT por parte del motor.

```
27 * CALL consultar_precio_producto('1');
28
29
30
```

Result Grid

 Filter Rows:

Export: 

Wrap Cell Content: 

	ID	Producto	Codigo_Barra	Marca	Categoria	Precio
	1	PRODUCTAZO1	PROD00000001	Gafa	Cuidado Personal	\$36.97

Evidencia de interacción con IA

Ver AnexoIA_TFI_BDI_Comision2_Grupo126_Orellana-Gomez-Galvan-Pagola.pdf
Etapa 4

Etapa 5. Concurrencia y transacciones

Simulación con dos sesiones MySQL para generar bloqueos y deadlocks

Implementación de la simulación: Ver script 08_transacciones.sql.

Se comienza dando permisos al usuario 'vendedor' para que pueda realizar transacciones desde su lado.

```
GRANT SELECT, UPDATE ON gestion_comercio.productos TO 'vendedor'@'localhost';  
SHOW GRANTS FOR 'vendedor'@'localhost';
```

Para poder demostrar un deadlock se utilizarán transacciones no commiteadas para que queden abiertas.

```
START TRANSACTION;  
UPDATE productos SET precio = precio + 10 WHERE producto_id = 1;
```

Se hace lo mismo del lado del usuario 'vendedor', se crea una transacción para actualizar un campo pero de otro id, en este caso 2.

```
START TRANSACTION;  
UPDATE productos SET precio = 100 WHERE producto_id = 2;
```

Finalmente se setean las consultas del lado del root y del 'vendedor' para ejecutar la prueba.

```
UPDATE productos SET precio = precio + 5 WHERE producto_id = 2; — desde el root  
UPDATE productos SET precio = precio + 8 WHERE producto_id = 1; — desde el usuario
```

El procedimiento es: Ejecutar el primer bloque de la primera transacción, luego ir a 'vendedor' y ejecutar el primer bloque de su transacción. Eso deja en ambos usuarios transacciones abiertas sin commitear afectando a los ids 1 y 2. A continuación, se ejecutan los updates de root (o vendedor) y en el otro usuario ejecutamos el update cruzado. El deadlock se da porque al ejecutar el segundo update en el primer usuario, como está intentando modificar el precio del id que ya está en una transacción, queda a la espera de que la transacción que tiene secuestrado el registro se resuelva, pero cuando vamos a la siguiente y hacemos update sobre el id que la primera está usando en su transacción, se produce el deadlock, ya que las dos quedan a la espera de la otra por terminar su operación sobre el registro. Esta operación también puede hacerse de forma que la alerta de deadlock se produzca sobre el otro usuario, simplemente hay que ejecutar el primer segundo update cruzado en el otro usuario primero.

El resultado, si las sentencias se ejecutan en el orden correcto es:

- Deadlock found when trying to get lock; try restarting transaction 5.437 sec

De esta prueba se concluye que las transacciones quedan abiertas hasta que las terminemos con commit, rollback o hasta que el motor termine una u otra por algún

motivo, en este caso deadlock. Mientras tanto, se puede hacer esto que se hizo anteriormente, ejecutar una parte del bloque, ir al otro usuario y hacer lo mismo para volver y ejecutar lo que queda de la sentencia. Esto revela mucho sobre el funcionamiento del motor.

Otra conclusión importante es que se puede deducir que una transacción al crearse tiene un nivel de insolation por defecto, al menos READ COMMITED, porque al crear la transacción no se dio ninguna especificación respecto al nivel de insolation pero de todas maneras el deadlock ocurrió, por lo que la transacción al menos no es READ UNCOMMITTED. Esto lo se verifica con la sentencia:

```
SELECT @@GLOBAL.tx_isolation, @@SESSION.tx_isolation;
```

Esto nos permite ver información sobre el tipo de insolation y se observa que es por defecto REPETEABLE READ, lo que supera en un nivel a READ COMMITED, por lo que la suposición era correcta.

Procedure y Handler para gestionar deadlocks

Se implementó el procedimiento `actualizar\_precio\_por\_id` con manejo de transacciones -
Ver 09 concurrencia guiada.sql (para la definición completa del procedimiento con handler para deadlocks).

```
DELIMITER $$
CREATE PROCEDURE actualizar_precio_por_id (IN precio_nuevo decimal(10,2), IN id INT)
BEGIN
    DECLARE contador INT DEFAULT 1;
    DECLARE realizado INT DEFAULT 0;

    WHILE contador <= 3 AND realizado = 0 DO
        BEGIN
            DECLARE EXIT HANDLER FOR 1213 -- , 1205, 1216 -- deadlock
            BEGIN
                SET contador = contador + 1;
                SELECT CONCAT('Deadlock capturado, contador ahora: ', contador) AS debug;
                IF contador <= 3 THEN
                    -- SELECT CONCAT('Deadlock, reintento ', contador) AS mensaje;
                    DO SLEEP(0.1 * contador);
                END IF;
            END;
            -- SELECT CONCAT('Intento ', contador, ' - Iniciando transacción') AS debug;
            START TRANSACTION;
            UPDATE productos SET precio = precio_nuevo WHERE producto_id = id;
            SET realizado = 1;
            COMMIT;
            -- SELECT CONCAT('Intento ', contador, ' - Éxito') AS debug;
        END;
        END WHILE;
        IF realizado = 1 THEN
            SELECT 'Actualización exitosa' AS resultado;
        ELSE
            SELECT 'Error: Falló después de 3 intentos' AS resultado;
        END IF;
    END $$
DELIMITER ;
```

Arriba se encuentra el procedimiento diseñado para manejar los deadlocks. La creación y declaración de variables dentro de un procedimiento ya están contempladas en las etapas anteriores, así que en esta oportunidad describiremos las herramientas y desafíos nuevos encontrados para desarrollar el script.

Para comenzar con lo importante vamos a hablar del Handler. El handler es un proceso que captura un error determinado, puede ser muy específico o amplio como lo vemos en los comentarios del código, que casi igual que try catch en java, interviene al momento de un error para ejecutar una acción o simplemente evitar que el error interrumpa el procedimiento. Las dificultades, además de la sintaxis, fueron comprender el flujo de la ejecución para determinar qué tipo de handler necesitábamos usar y lo contraintuitivo que puede ser el flujo de sql.

Dentro de los Handlers experimentamos con exit y continue. Continue captura el error, ejecuta su bloque de sentencias y continúa la ejecución del código a partir del punto en el que el error ocurrió sin volver a evaluar con while y continuando con el código a pesar del error. Este no es el escenario que necesitamos porque necesitamos que el handler pase a la siguiente iteración.

Exit por otro lado secuestra el spotlight al capturar el error en UPDATE pero en este caso luego de ejecutar su bloque, sale del BEGIN-END interno y esto es clave porque así el WHILE puede volver a empezar el intento.

Dentro del handler tenemos un bloque que ejecuta simplemente un aumento en el contador que el iterador evalúa y usa un DO SLEEP para dar un tiempo de espera hasta la próxima ejecución de la transacción. Finalmente encontramos la transacción que es motivo del procedimiento.

Cabe destacar que intentamos probar la captura de un deadlock con el handler capturando el error 1213, simulando la situación del punto anterior donde hicimos consultas cruzadas apuntando a registros siendo usados por transacciones abiertas desde dos usuarios distintos, pero no logramos causar un deadlock (error 1213) llamando al procedure desde dos sesiones. A diferencia de cuando lo hicimos manualmente, donde controlamos cada paso, el procedure hace todo (START, UPDATE, COMMIT) muy rápido de una vez. Esto hizo muy difícil coordinar las dos llamadas para que justo se bloquearan mutuamente. Pero para probar que el handler sí atrapaba errores y que el bucle WHILE reintentaba como queríamos, cambiamos el handler para que atrape el error de timeout (1205).

Para esto seteamos el tiempo máximo que la transacción espera por un lock usando:

```
SET SESSION innodb_lock_wait_timeout = 3; -- setear el tiempo
```

Luego agregamos el código 1205 al handler, lo dropeamos y volvemos a crear. Luego ejecutamos una transacción abierta desde el 'vendedor' haciendo un UPDATE a id = 2 y desde el root llamamos al procedimiento así:

```
CALL actualizar_precio_por_id(12,2); -- llamamos al procedimiento
```

Esto se ejecuta y devuelve 4 rows imprimiendo los debuggers dentro del procedimiento:

- 'Deadlock capturado, contador ahora: 2'
- 'Deadlock capturado, contador ahora: 3'
- 'Deadlock capturado, contador ahora: 4'
- 'Error: Falló después de 3 intentos'

Este resultado muestra que:

- 1) El handler (que pusimos para el error 1205) sí se activó cada vez que falló el UPDATE.
- 2) El contador subió correctamente hasta 4 (falló 3 veces).
- 3) El SLEEP funcionó.
- 4) Después del tercer intento fallido, el WHILE terminó y mostró el mensaje de error final. Con esto validamos que la lógica de reintentos funcionaba.

Pruebas de comparación de distintos niveles de insolation

Para comparar los niveles de aislamiento READ COMMITTED y REPEATABLE READ usamos un proceso simple.

Primero hay que aclarar que es lo que esperamos de cada nivel de aislamiento respecto a lo que difieren fundamentalmente. El nivel READ COMMITTED permite lecturas no repetibles y REPEATABLE READ no permite lecturas no repetibles.

La lectura no repetible se refiere a que una transacción en el nivel READ COMMITTED que lee un dato durante su ejecución pero antes de que termine este cambia y lo vuelve a leer, lo actualiza y muestra o usa el dato actualizado. A esto se le llama lectura no repetible, porque no puede repetir el valor original de la lectura.

Por otro lado REPEATABLE READ en la misma situación cuando lee por segunda vez el dato no cambia el valor que tiene almacenado por lo que puede enseñar el valor original. A esto se le dice que NO permite lectura no repetible, es decir que la lectura si es repetible y puede mostrar o usar el valor original del dato.

Para probar esto ejecutamos el siguiente script desde el root y modificamos el valor desde otra transacción en 'vendedor'.

Primero consultamos el valor del dato de prueba.

```
SELECT precio FROM productos WHERE producto_id = 5; -- devuelve 815.82
```

A continuación creamos una transacción con la misma consulta anterior y asignamos el nivel de aislamiento READ COMMITTED. Ejecutamos primero el script desde el inicio hasta el SELECT y dejamos la transacción abierta. El resultado de la consulta es precio = 815.82.

```
SET SESSION TRANSACTION ISOLATION LEVEL READ COMMITTED;  
START TRANSACTION;  
SELECT precio FROM productos WHERE producto_id = 5; - etapa 1. Inicio de etapa 2  
COMMIT;- fin de etapa 2
```

Luego vamos a el usuario 'vendedor' y ejecutamos la transacción que nos modifica el valor del precio del producto id 5.

```
START TRANSACTION;  
UPDATE productos SET precio = precio + 100 WHERE producto_id = 5;  
COMMIT;
```

Después volvemos al root y ejecutamos la segunda mitad de la transacción que dejamos abierta desde el SELECT hasta COMMIT. En esta oportunidad la lectura del precio nos devuelve 915.82. Esto demuestra que el dato precio tuvo dos valores durante la ejecución de la transacción.

Para comprobar el mismo caso en REPEATABLE READ usamos el siguiente script.

```
UPDATE productos SET precio = 815.82 where producto_id = 5; -- devolvemos el producto a su precio original  
SELECT precio FROM productos WHERE producto_id = 5; -- devuelve 815.82 nuevamente  
SET SESSION TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
START TRANSACTION; -- en ese caso usamos el nivel de aislamiento REPEATABLE READ. Viene por defecto pero para claridad usamos SET.  
SELECT precio FROM productos WHERE producto_id = 5; -- hacemos el mismo proceso y ejecutamos la transacción hasta acá y vamos a s2 a ejecutar la transacción  
-- leemos 815.82
```


COMMIT; - - aquí seguimos leyendo 815.82

Realizamos el mismo proceso que anteriormente pero en este caso la lectura de la segunda parte del script nos devuelve el mismo valor que leímos originalmente, como esperábamos de este nivel de aislamiento.

Sobre transacciones y concurrencia

Durante esta etapa trabajamos con deadlocks y requirieron entender el orden en que se ejecutan las operaciones de consulta o modificación durante la concurrencia para poder lograr un deadlock. Esto lo trasladamos a procedures para poder gestionar deadlocks sin romper la ejecución y para esto tuvimos que generar bloques lógicos con BEGIN-END y usar handler para atrapar las excepciones, dando una nueva (y dificultosa) perspectiva sobre cómo es el flujo de ejecución dentro de un procedure. Por último vimos las diferencias en la práctica entre distintos tipos de niveles de aislamiento dejando claro la importancia de saber elegir cual usar dependiendo de las necesidades.

Conclusiones

Este trabajo demostró competencias integrales en diseño, implementación, optimización y seguridad de bases de datos. Se logró un sistema robusto que maneja eficientemente grandes volúmenes de datos, con medidas de seguridad apropiadas y manejo profesional de concurrencia.